Decrease and Conquer

One smaller problem at a time

Constant decrease • constant-factor decrease • variable-size decrease

CORE IDEA

n → n - 1

n → n / 2

n → n - k

Reduce → solve one smaller instance → extend

DECREASE & CONQUER

Learning outcomes

By the end of this lecture, you should be able to:

EXPLAIN

the decrease-and-conquer design strategy

DISTINGUISH

decrease-by-one, decrease-by-factor and variable-size decrease

CLASSIFY

Insertion Sort and discuss Selection/Bubble as related interpretations

APPLY

Binary Search as a decrease-by-factor algorithm

TRACE

Euclid’s algorithm as variable-size decrease

CONNECT

graph traversal, topological sorting and combinatorial generation

COMPARE

brute force, decrease-and-conquer and divide-and-conquer

Where this topic fits

Official sequence: Brute Force → Decrease and Conquer → Divide and Conquer

BRUTE FORCE

Systematically examine candidates or straightforward possibilities.

→

DECREASE & CONQUER

Reduce to ONE smaller instance, solve it, then extend the result.

→

DIVIDE & CONQUER

Split into TWO OR MORE smaller subproblems, solve them, then combine.

Today: “one smaller instance” is the defining idea.

The core pattern

Decrease and Conquer solves only one smaller instance.

1 DECREASE

Transform the original problem into one smaller instance.

2 CONQUER

Solve that smaller instance, often recursively or iteratively.

3 EXTEND

Use the smaller solution to construct the solution to the original problem.

Decrease vs. Divide and Conquer

The number of recursive subproblems is the key distinction.

DECREASE AND CONQUER

One smaller problem

T(n) = T(smaller n) + work

Examples:
• Insertion Sort
• Binary Search
• Euclid’s GCD

DIVIDE AND CONQUER

Two or more smaller problems

T(n) = aT(n/b) + work, a > 1

Examples:
• Merge Sort
• Quick Sort
• Maximum Subarray (classic D&C)

ONE smaller instance ≠ MULTIPLE smaller instances

Algorithms can have several classifications

These labels describe different properties - they are not mutually exclusive.

IN-PLACE

How much extra memory?

STABLE

What happens to equal keys?

COMPARISON-BASED

How are keys ordered?

DESIGN STRATEGY

How is the problem structurally solved?

Insertion Sort = in-place + stable + comparison-based + decrease-and-conquer

Design choice or side effect?

A smaller active region does not automatically make an algorithm decrease-and-conquer.

DECREASE BY DESIGN

The algorithm is deliberately formulated as:

“solve one smaller instance, then extend its solution.”

The reduction is central to the reasoning and recurrence.

SHRINKS WHILE RUNNING

The active region may become smaller after a pass, but the shrinkage may simply be a consequence of the procedure rather than its defining design idea.

Ask: “Is the smaller problem the design idea, or merely what happens next?”

Three forms of Decrease and Conquer

The smaller instance can be created in three standard ways.

1 DECREASE BY A CONSTANT

Typical form:

$n \rightarrow n - 1$

The problem shrinks by the same fixed amount each step.

Classic example: Insertion Sort.

2 DECREASE BY A FACTOR

Typical form:

$n \rightarrow n / 2$

A fixed fraction of the remaining problem is discarded each step.

Classic example: Binary Search.

3 VARIABLE-SIZE DECREASE

Typical form:

$n \rightarrow n - k$

The amount removed depends on the current input or result.

Classic example: Euclid’s GCD.

Decrease by a constant amount

The most common form is decrease-by-one.

$$n \rightarrow n-1 \rightarrow n-2 \rightarrow \ldots \rightarrow 1$$

WHAT HAPPENS?

Each step solves a problem whose size is smaller by a fixed constant, usually 1.

WHY USE IT?

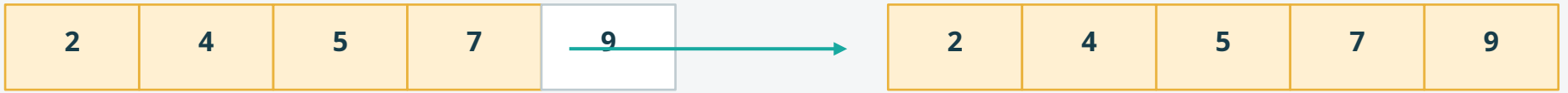
The solution to size n can often be obtained by extending a solution already known for size n - 1.

EXAMPLE

Sort the first n - 1 items, then place the nth item correctly: Insertion Sort.

Insertion Sort is decrease-and-conquer by design

The “smaller solved instance” is the sorted prefix.



sorted first n - 1 elements

insert 9 into its correct position

Sort(n) = Sort(n - 1) + insert the remaining key

The reduction n → n - 1 is not accidental; it is the structure of the algorithm.

Insertion Sort trace

Example: sort [7, 3, 8, 2, 6]. The sorted prefix grows one item at a time.



Start: first element is trivially sorted



Insert 3 into [7]



Insert 8 into [3,7] - no shift needed



Insert 2 into [3,7,8] - shift 8,7,3



Insert 6 into [2,3,7,8] - shift 8,7

Insertion Sort recurrence

The reduction is by one, but inserting may scan the sorted prefix.

$$T(n) = T(n - 1) + \Theta(n)$$

BEST CASE

Already sorted input. Each new key needs only a quick comparison.

$\Theta(n)$

AVERAGE CASE

A key shifts through part of the sorted prefix on average.

$\Theta(n^2)$

WORST CASE

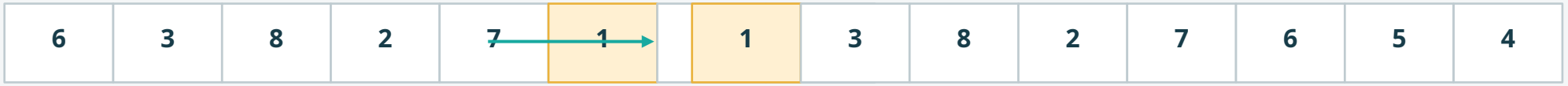
Reverse-sorted input. Each key shifts across the whole prefix.

$\Theta(n^2)$

Decrease-and-conquer does not automatically mean “fast”; the extension work matters.

Selection Sort: a reasonable decrease-by-one interpretation

After selecting the minimum, one position is permanently solved.



Find minimum = 1

Position 0 is now final

WHY IT FITS

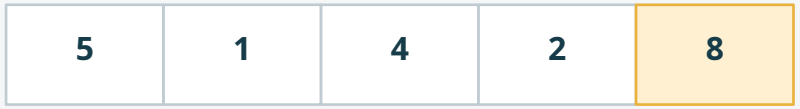
Once the minimum is fixed, the remaining sorting problem has size n - 1. This makes a recursive decrease-by-one interpretation perfectly valid.

WHY WE WORD IT CAREFULLY

Selection Sort is often introduced as a straightforward iterative/basic sorting algorithm; decrease-and-conquer is a useful interpretation rather than always the headline classification.

Bubble Sort: the active region shrinks too

But the shrinkage is usually not treated as its central design paradigm.



After one full pass, the largest item is fixed at the end.

active unsorted region: n → n - 1

TEACHING POINT

A shrinking active region can happen as a consequence of a pass. Decrease-and-conquer is clearest when “solve one smaller instance” is the actual design idea used to formulate the algorithm.

Three simple sorts - one useful distinction

All are in-place; their relationship to decrease-and-conquer is not equally strong.

Algorithm	In-place?	How the active problem changes	Decrease-and-conquer view
Insertion Sort	Yes	Sorted prefix of size n - 1 is extended	Strong / textbook
Selection Sort	Yes	Fix minimum, continue on remaining n - 1	Reasonable interpretation
Bubble Sort	Yes	Largest settles; unsorted region becomes n - 1	Possible but weaker framing

Classification depends on what property you are describing.

Decrease by a constant factor

Instead of removing one item, discard a fixed fraction of the remaining problem.

$$n \rightarrow n/2 \rightarrow n/4 \rightarrow n/8 \rightarrow \ldots \rightarrow 1$$

WHY THIS CAN BE POWERFUL

A constant-factor reduction reaches size 1 after only about $\log_2 n$ steps. Each decision removes a large part of the remaining search space.

CLASSIC EXAMPLE

Binary Search uses ordering information to decide which half cannot contain the target, then continues with only the other half.

The problem does not shrink by itself

Same searching problem - different algorithmic choices.

Problem: Given a collection and a target x, determine whether x is present.

BRUTE FORCE CHOICE

Check candidates one by one.

No structural elimination is required.

Linear Search: O(n)

DECREASE-AND-CONQUER CHOICE

If the data is sorted, compare with the middle and deliberately discard half.

Binary Search: O(log n)

The algorithm chooses how to reduce the remaining work.

Brute-force search: Linear Search

Example: find 25 in an unsorted list.

12	7	19	3	25	8	14

12 ≠ 25	7 ≠ 25	19 ≠ 25	3 ≠ 25	25 = 25 ✓

Worst case: inspect all n elements → Θ(n)

Straightforward and valid even when the data has no useful ordering.

Binary Search needs structure

The faster reduction is available only because the data is ordered.

ONE SEARCH ON UNSORTED DATA

If the list is unsorted and you only need one lookup, sorting first may cost more than a simple linear scan.

Linear Search: Θ(n)
Sort + Binary Search: typically Θ(n log n) + Θ(log n)

MANY SEARCHES ON SORTED DATA

If the data is already sorted, or preprocessing is worthwhile for many queries, Binary Search can repeatedly exploit the order.

Each search: Θ(log n)

A better asymptotic search method may require a stronger input assumption.

Binary Search: decrease by a factor of 2

Compare with the middle and keep only the half that can still contain the target.

3	7	8	12	14	19	25	31	40	56

middle = 14

Target = 25 > 14 → discard the entire left half

10 elements → about 5 elements

Binary Search trace - part 1

Search for 25 in [3, 7, 8, 12, 14, 19, 25, 31, 40, 56].

3	7	8	12	14	19	25	31	40	56

Step 1: middle = 14. Since 25 > 14, keep only the right half.

19	25	31	40	56

Step 2: middle = 31. Since 25 < 31, keep only the left part.

Binary Search trace - part 2

Each comparison deliberately discards a whole region.



Step 3: middle = 19. Since 25 > 19, keep the right item.



Step 4: 25 = 25. Found.

10 → 5 → 2 → 1

Why Binary Search is logarithmic

The problem size is halved at each recursive step.

$$T(n) = T(n/2) + \Theta(1)$$

After k reductions: $n / 2^k = 1 \Rightarrow 2^k = n \Rightarrow k = \log_2 n$

$$T(n) = \Theta(\log n)$$

A constant-factor reduction reaches the base case very quickly.

Linear Search vs. Binary Search

Same basic problem; different assumptions and design strategies.

Property	Linear Search	Binary Search
Input requirement	No ordering required	Data must be sorted
Design view	Straightforward / brute-force scan	Decrease by factor
Reduction per step	Usually one candidate examined	About half discarded
Worst-case time	Θ(n)	Θ(log n)
Best use	Small / unsorted / one-off search	Repeated search on sorted data

Variable-size decrease

The amount removed is determined by the current state of the problem.

$$n \rightarrow n - k \quad \text{where } k \text{ is not fixed}$$

WHAT MAKES IT DIFFERENT?

There is no guaranteed “minus one” or “divide by two” pattern. The next smaller instance depends on a value computed from the current input.

CLASSIC EXAMPLE

Euclid’s algorithm replaces (a, b) with (b, a mod b). The remainder determines how much the problem shrinks.

Euclid’s algorithm for GCD

Repeatedly replace the pair with a smaller pair until the remainder becomes zero.

PSEUDOCODE

```
GCD(a, b)
while b ≠ 0
    r ← a mod b
    a ← b
    b ← r
return a
```

WHY DECREASE-AND-CONQUER?

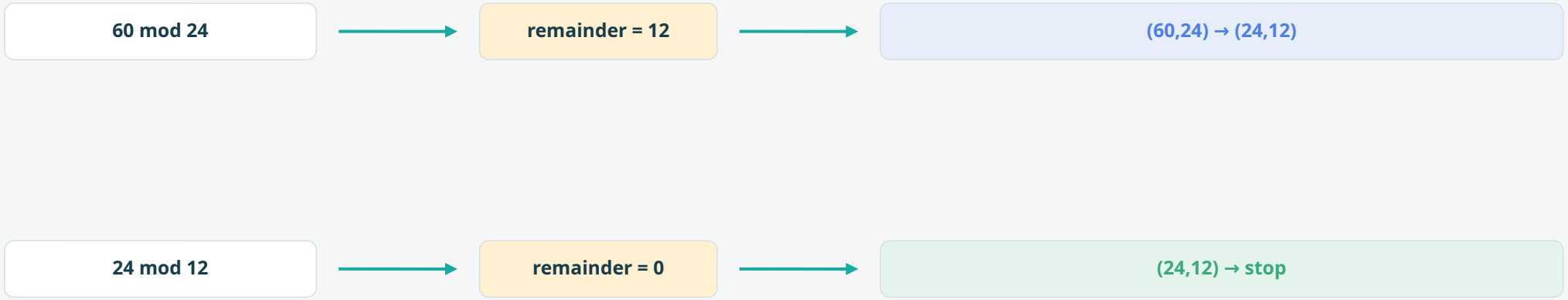
Each step keeps only one smaller instance: (b, a mod b).

The new size is controlled by the remainder, so the decrease is variable rather than fixed.

(a, b) → (b, a mod b)

Euclid trace

Find GCD(60, 24).



GCD(60, 24) = 12

Why Euclid is “variable-size”

The next problem size is controlled by the remainder, not a fixed rule like n - 1 or n/2.

Step	Current pair	Remainder	Next pair
1	(1071, 462)	147	(462, 147)
2	(462, 147)	21	(147, 21)
3	(147, 21)	0	Stop

Notice: the reduction is 1071 → 462 → 147 → 21. The amount removed changes every time.

CDF connection: graph traversal

The official topic list also connects Decrease and Conquer with traversal-style algorithms.

TRAVERSAL VIEW

At each step, process one previously unvisited vertex and continue with the remaining unvisited structure. The explored/unexplored boundary changes as the algorithm progresses.

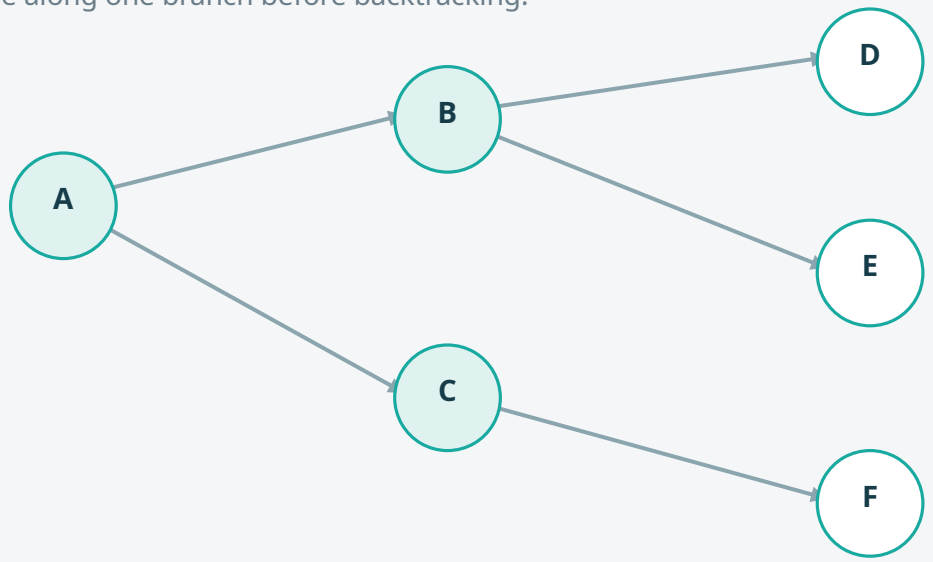
IMPORTANT NUANCE

DFS and BFS are primarily taught as graph traversal algorithms. The decrease perspective is a way to see progressive reduction of the unprocessed portion, not the only or universal classification.

Process one part → continue on what remains

Depth-First Search

Explore as far as possible along one branch before backtracking.



ONE POSSIBLE DFS ORDER

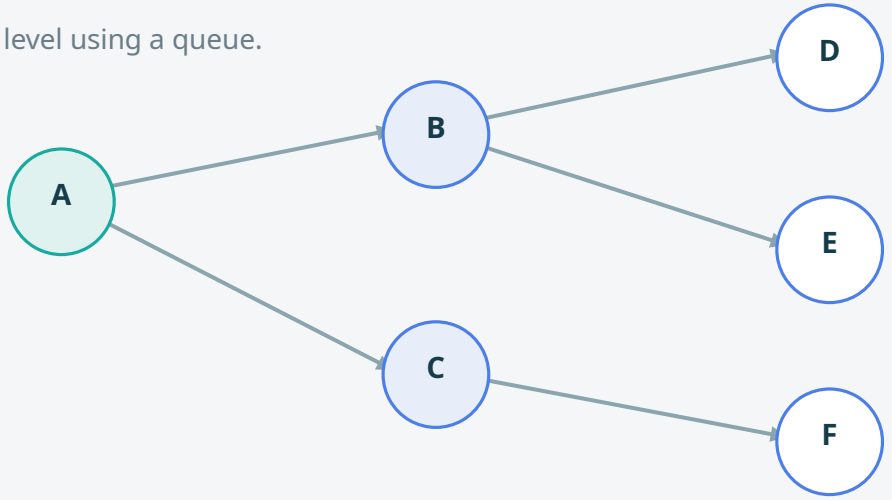
A → B → D → E → C → F

At each step, one new vertex becomes processed; recursion/stack controls which remaining branch is explored next.

Traversal is included here because it progressively reduces the unprocessed graph, but it is best remembered first as a graph algorithm.

Breadth-First Search

Explore vertices level by level using a queue.



BFS ORDER

A → B → C → D → E → F

The queue stores discovered vertices waiting to be processed. Each processing step reduces the set of unprocessed reachable vertices.

DFS: stack/recursion **BFS: queue**

Topological Sorting

For a DAG, repeatedly choose a source vertex and remove it from the remaining problem.

SOURCE-REMOVAL IDEA

A source has indegree 0.

1. Choose a source.
2. Output it.
3. Remove it and its outgoing edges.
4. Repeat on the smaller DAG.

WHY IT FITS NICELY

Each removal leaves exactly one smaller instance of the same problem: topologically sort the remaining DAG.

This is a clear decrease-by-one view.

n vertices → n - 1 vertices

Kahn’s algorithm

A practical topological-sort implementation using indegrees and a queue.

PSEUDOCODE

```
compute indegree of every vertex
put all indegree-0 vertices in queue

while queue not empty:
    u ← remove from queue
    output u
    for each edge u → v:
        indegree[v]--
        if indegree[v] = 0:
            add v to queue
```

DECREASE VIEW

Every time a vertex is output, it is permanently removed from the unresolved DAG.

The remaining task is the same type of problem on one smaller graph.

If all vertices cannot be removed, the graph contains a cycle.

Topological Sort trace

DAG edges: A→C, B→C, B→D, C→E, D→E.

Step	Available sources	Choose	Output so far
Start	A, B	A	A
2	B	B	A, B
3	C, D	C	A, B, C
4	D	D	A, B, C, D
5	E	E	A, B, C, D, E

Each chosen source reduces the unresolved graph by one vertex.

Generating combinatorial objects

Another CDF item: construct size n objects from already-generated size n - 1 objects.

GENERAL IDEA

Generate the objects for n - 1 items.

Then extend each smaller object in every valid way to include the nth item.

EXAMPLE: PERMUTATIONS

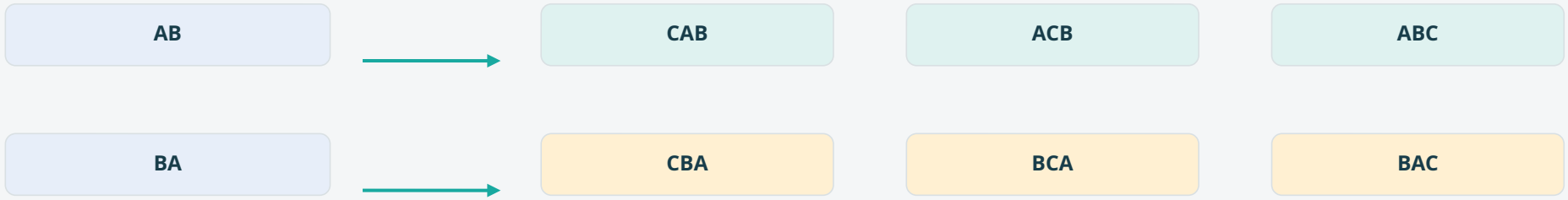
Start with permutations of {A, B}.

Insert C into every possible position to obtain the permutations of {A, B, C}.

size n - 1 solutions → extend → size n solutions

Permutation generation by decrease-and-conquer

Build permutations of {A, B, C} from permutations of {A, B}.



Insert C into every possible position.

$$P(n) = n \times P(n - 1) \Rightarrow n! \text{ permutations}$$

The number of outputs grows factorially even though the construction uses a smaller-instance idea.

Three variants - compare them

Recognize the reduction rule before analyzing the recurrence.

Variant	Typical reduction	Representative example	Typical recurrence
Constant amount	n → n - 1	Insertion Sort	T(n)=T(n-1)+work
Constant factor	n → n/2	Binary Search	T(n)=T(n/2)+work
Variable size	depends on current input	Euclid’s GCD	T(state)=T(smaller state)+work

All three solve only ONE smaller instance.

How to recognize Decrease and Conquer

Ask these questions in order.

1. **Is the original problem reduced to a smaller instance of the same problem?**
2. **Do we solve only ONE of those smaller instances?**
3. **Can the original answer be obtained by extending that smaller solution?**
4. **How is the size reduced: by 1, by a factor, or by a variable amount?**

Common misconceptions

Avoid these shortcuts when classifying algorithms.

“IT SHRINKS, SO IT MUST BE D&C”

No. Many iterative algorithms have a shrinking active region. The reduction must be part of the design strategy.

“DECREASE = DIVIDE”

No. Decrease and Conquer solves one smaller instance. Divide and Conquer solves multiple smaller subproblems.

“DECREASE MEANS FAST”

No. Insertion Sort decreases by one but can still take $\Theta(n^2)$. Complexity depends on both the reduction and the work done at each step.

Design technique ≠ complexity class

Checkpoint: classify the reduction

For each algorithm, identify the most natural reduction pattern.

Item	Algorithm	Your classification
A	Insertion Sort	?
B	Binary Search	?
C	Euclid’s GCD	?
D	Merge Sort	?
E	Selection Sort	?

Think before revealing the next slide.

Checkpoint answers

The wording matters for borderline cases.

Algorithm	Best classification here	Reason
Insertion Sort	Decrease by 1	Sort n - 1, then insert one key
Binary Search	Decrease by factor	Discard about half each step
Euclid’s GCD	Variable-size decrease	Remainder controls next size
Merge Sort	Divide and Conquer	Solves two halves
Selection Sort	Decrease-by-1 interpretation	Fix one minimum, continue on n - 1

Practice: what does the recurrence tell you?

Identify the design pattern before solving the recurrence.

$T(n)=T(n-1)+n$	one smaller problem; decrease by 1
$T(n)=T(n/2)+1$	one smaller problem; decrease by factor
$T(n)=2T(n/2)+n$	**two smaller problems; divide and conquer**
$T(n)=T(n-r)+1$	one smaller problem; variable-size decrease if r varies

A problem that exposes the limit of “one smaller instance”

Maximum Subarray: find the contiguous subarray with the largest sum.



Best subarray = [4, -1, 2, 1] sum = 6

BRUTE FORCE

Try many candidate subarrays and keep the best. Straightforward, but expensive.

QUESTION

Can we exploit the structure of the array instead of enumerating every candidate?

Maximum Subarray by brute force

Enumerate start/end positions and compute each candidate sum.

number of contiguous subarrays = n(n + 1) / 2

$O(n^3)$ VERSION

Choose start i.
Choose end j.
Sum A[i..j] with another loop.

Three nested loops.

$O(n^2)$ VERSION

Choose start i.
Extend end j while maintaining a running sum.

Two nested loops.

WHAT WE WANT NEXT

A structural strategy that avoids examining every candidate subarray explicitly.

This is a natural place to move beyond brute force.

Why Decrease and Conquer is not the natural fit here

Removing one item does not cleanly preserve where the optimal subarray might lie.

TRY TO REMOVE ONE ELEMENT

If we solve the maximum-subarray problem on n - 1 elements, adding the removed element back can change the best answer in complicated ways. The optimum may start, end, or cross near that position.

A BETTER QUESTION

Instead of removing one element, what if we split the array into two meaningful regions and reason about where the optimum can occur?

Maybe ONE smaller instance is not enough.

Split the array in half

The maximum subarray must fall into one of three cases.

-2	1	-3	4	-1	2	1	-5	4

CASE 1

Entirely in the LEFT half.

Solve the left half.

CASE 2

Entirely in the RIGHT half.

Solve the right half.

CASE 3

CROSSES the midpoint.

Compute the best crossing sum.

Now we need more than one smaller subproblem.

This becomes Divide and Conquer

Two recursive halves plus linear work for the crossing case.

$$T(n) = 2T(n/2) + \Theta(n)$$

LEFT

Maximum subarray entirely in left half.

RIGHT

Maximum subarray entirely in right half.

CROSSING

Best suffix of left + best prefix of right.

Master Theorem ⇒ $\Theta(n \log n)$

Recap and bridge to the next design technique

Decrease and Conquer teaches us to exploit ONE smaller instance; some problems demand several.

DECREASE BY 1

Insertion Sort; Selection Sort can be interpreted similarly

DECREASE BY FACTOR

Binary Search discards about half each step

VARIABLE DECREASE

Euclid’s GCD uses the remainder to determine the next size

CDF CONNECTIONS

Graph traversal, topological sorting, combinatorial generation

KEY DISTINCTION

Problem shrinkage should be part of the design idea, not merely a side effect

NEXT

Divide and Conquer: Merge Sort, Quick Sort, Maximum Subarray

Decrease and Conquer = solve ONE smaller instance.